

Computer Architecture (Basics)

Computer Architecture defines the conceptual design & fundamental operational structure of a computer system.

- It acts as a **functional blueprint** for a programmer, specifying what the hardware can do.

Architecture vs organization vs Assembly language

• What the machine does (interface)	• How the machine implements it. (Hardware, circuitry, bus, etc.)	• How the Humans instruct the interface. (software).
• High level functional blueprint	• low level physical micro architecture	• Human-readable representation of machine code.
• e.g: CPU supports a 32 bit add function using x1 & x2 registers.	• e.g CPU uses a 5-stage pipeline & a carry-lookahead adder to complete instructions.	• add x1, x2;

Difference between RISC & CISC architecture:

Before in ~~an~~ COAL course we learned ~~mas~~ MASM which is CISC architecture.

Now we will ~~&~~ learn RISC ~~&~~ architecture.

Main Difference:

RISC keeps instructions simple to optimise execution speed per cycle.

CISC provides complex instructions to minimize the number of lines of code needed per program.

Page

day/date

Instruction difference example:

In **CISC** e.g. x86 for ADD we had
So many ~~rules~~ different variants.

ADD ax, bx

ADD mem, ax

ADD ax, mem

- - - -

- - - .

- - - -

But in **RISC** we will only have ONE

ADD ax, bx, cx.

So it is a very simplified version.

A little bit about organization:

(How it is decided what ^{max} amount of RAM can be placed in a PC).

¶ We already know that there are different buses:

- 1) Data bus: transfers actual data
- 2) Address bus: ~~the~~ tells the address where data is stored.
- 3) Control bus

Now if the Address bus size is ~~8~~ 8-bit, then it can refer to max of 2^8 addresses. so only 256 bytes.

Thus if our Address bus is 8-bit we can have a maximum RAM of 256 bytes.

If our Address bus is ~~32~~ 32-bit, it can refer to 2^{32} addresses so 4096 MB or 4GB RAM.

so our RAM size depends upon our Address bus.

Secondly Data bus determines how many cycles would it take to bring the data.

e.g. an int is 32 bit but our data bus is 16-bit, it will take 2 cycles but if our data bus was 64-bit it can even transfer 2 ints in one go!

Measuring Performance

Consider we say one computer is better than other in performance. But what do we even mean by this statement?

Performance can be measured by different metrics. e.g consider an airplane, its metrics can be speed, range & capacity.

But still consider one metric like speed of an airplane. Now consider 2 plane: one plane transports 2 passengers but in just 30 minutes, second plane transports 480 passengers but in 1.5 hrs. Now which is better?

Like with this same analogy applies to computers as well.

Performance depends heavily on the user.

A single user waiting for a program to finish cares about response time while a system administrator manager managing a server farm cares about through put time.

Response Time:

also called execution time.

The total time required for a computer to complete a task, including disk access, memory access, I/O activities, OS overhead CPU execution time etc.

Through put Time:

also called band width.

It is the number of tasks completed per unit time.

Performance:

Overall system performance for a program is defined as:

$$\text{Performance } x = \frac{1}{\text{response time } x.}$$

OR.

$$\text{performance } x = \frac{1}{\text{execution time } x.}$$

* relation is inversely proportional.

Comparing performance of 2 systems:

$$\frac{\text{performance } x}{\text{performance } y} = \frac{\text{execution time } y}{\text{execution time } x.}$$

if n is > 1 , x has higher performance
 if n is < 1 , y has higher performance
 if n is $= 1$, same performance

Wall Clock time vs CPU Time:

~~CPU~~ **CPU Time** is actual time spent by the CPU actively calculating instructions for your specific program. ~~It includes time spent waiting for other apps~~

Wall clock time or Elapsed time: ~~is~~ ~~the~~ ~~total~~ ~~time~~ ~~to~~ ~~finish~~ ~~a~~ ~~task~~. It includes CPU time & time spent executing other OS tasks.

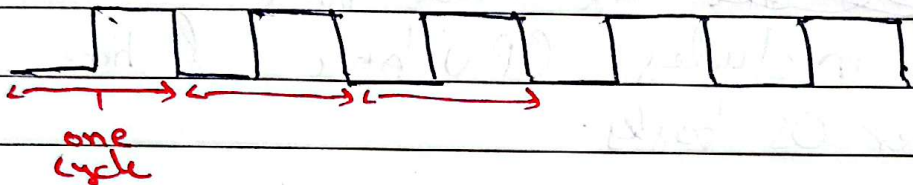
~~Elapsed time~~
CPU Time further

User CPU Time: The CPU time spent in a program itself.

System CPU Time: The CPU time spent in the OS performing task on behalf of program

Time & clock metrics:

All computers are driven by an internal clock that beats at a constant rate synchronizing every hardware event in the processor.



clock ~~is~~ beating

Now we will need to learn a few clock metrics because they are used in measuring performance.

Clock Cycle: This is the time for one clock period, which runs at constant rate. Also called *tick, clock tick, period*.

Clock Period: Length of each clock cycle. ~~also called clock cycle time.~~ also called clock cycle time.
Unit: seconds (s)

Clock Rate: The number of clock cycles per second. (frequency).
unit = Hz (Hertz).

$$\text{clock cycle time} = \frac{1}{\text{clock rate (frequency)}}$$

Calculating execution time:

$$\text{CPU execution time for a program} = \text{CPU clock cycles for a program} \times \text{Clock cycle time.}$$

$$\text{CPU execution time for a program} = \frac{\text{CPU clock cycles for a program}}{\text{Clock rate.}}$$

Example: Question: A program runs in 10 seconds on computer A which has 2 GHz clock. We are trying to design a computer B that will run program in 6 seconds. ~~The~~ You have seen that we can increase the clock rate but due to this design computer B requires 1-2 times as many clock cycles as A for the program. What will be the clock rate ~~rate~~ required for B?

computer A:

CPU time taken = 10 second.

Clock ~~rate~~ rate = 2 GHz.

of ~~CPU~~ clock cycles = ?

comp B:

CPU time = 6 second.

Clock rate = ?

of ~~CPU~~ clock cycles ~~rate~~ = $1.2 \times A's \text{ cycles}$.

~~Ans~~

$$\text{CPU time of A} = \frac{\text{\# of cycles of A}}{\text{clock rate of A.}}$$

$$\begin{aligned}\text{\# of cycles of A} &= 10 \times (2 \times 10^9) \\ &= 20 \times 10^9 = \del{20 \times 10^9}\end{aligned}$$

$$\begin{aligned}\text{\# of cycles of B} &= 1.2 \times 20 \times 10^9 \\ &= 24 \times 10^9.\end{aligned}$$

$$\text{CPU time of B} = \frac{\text{\# of cycles of B}}{\text{clock rate of B}}$$

$$6 = \frac{24 \times 10^9}{\text{clock rate of B}}$$

$$\begin{aligned}\text{clock rate of B} &= 4 \times 10^9 \\ &= 4 \text{ GHz}\end{aligned}$$

← answer

Instruction Performance:

The previous equations did not refer to the number of instructions needed for the program. But since a compiler generates instructions to execute, the execution time must depend upon number of instructions.

Example:

consider A & B do the same thing but with different instructions:

A	B
Add	MUL
Add	MUL
MUL	SUB.
INC	
SUB	
ADD	
# of instruction = 6	# of instruction = 3.

now consider all operations take 1 cycle
but MUL take 10 cycles.

so

A takes 15 cycles
B takes 21 cycles.

Average cycle Per instruction of A = $\frac{15}{6} = 2.5$ sec.

Average cycle Per instruction of B = $\frac{21}{3} = 7$ seconds.

(Average)

Clock cycles per instruction: is the average number of clock cycles each instruction takes to execute. it

• it is abbreviated as **CPI**

Number of clock cycles:

Using CPI we can also write number of clock cycles in its format.

$$\# \text{ of clock cycles} = \text{Total instructions} * \text{CPI}.$$

= Total instruction is also written as Instruction Count (IC).

Execution time,

Now by substituting # of clock cycles by instruction, (above formula) we can get execution time in terms of ~~clock~~ instructions.

$$\text{Execution time} = \frac{\text{Instruction Count} * \text{CPI}}{\text{clock rate}}.$$

$$\text{Execution time} = \text{Instruction Count} * \text{CPI} * \text{clock cycle time}.$$